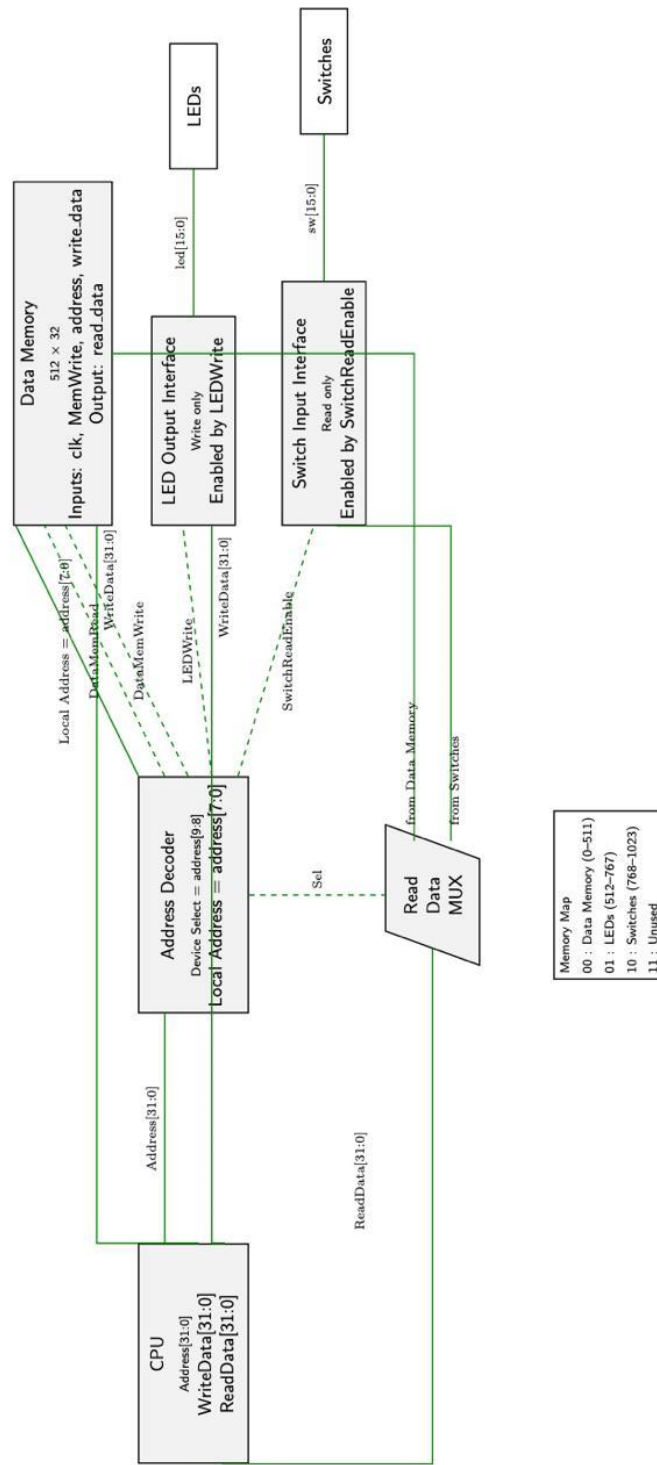


Syed Sualeh abbas, Syed Zaid Naqvi, Mohsin Mujtaba

Lab 8: Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Task 1: Design of Data Memory and Address Decoder

Block diagram of memory system with address decoding.



Task 2: Verilog Implementation and Testbench Development

Verilog source file for Data Memory

```
module DataMemory (  
    input    clk,  
    input    MemWrite, // Write enable (synchronous)  
    input    MemRead,  // Read  enable (asynchronous)  
    input [8:0] address, // 9-bit local address → 512 locations  
    input [31:0] write_data,  
    output reg [31:0] read_data  
);  
  
    // 512-word × 32-bit memory array  
    reg [31:0] memory [0:511];  
  
    // Initialise all locations to 0 (helps simulation)  
    integer i;  
    initial begin  
        for (i = 0; i < 512; i = i + 1)  
            memory[i] = 32'b0;  
    end  
  
    // Synchronous write  
    always @(posedge clk) begin  
        if (MemWrite)  
            memory[address] <= write_data;  
    end  
  
    // Asynchronous (combinational) read  
    always @(*) begin  
        if (MemRead)  
            read_data = memory[address];  
        else  
            read_data = 32'b0;  
    end  
  
endmodule
```

Verilog source file for Address Decoder

```
module AddressDecoder (  
    input [31:0] address,  
    input      readEnable,  
    input      writeEnable,  
    output reg  DataMemWrite,  
    output reg  DataMemRead,  
    output reg  LEDWrite,  
    output reg  SwitchReadEnable  
);  
  
    // Only the top 2 bits of the 10-bit local address determine the device  
    wire [1:0] deviceSelect;  
    assign deviceSelect = address[9:8];  
  
    always @(*) begin  
        // Safe defaults - only one device active at a time  
        DataMemWrite  = 1'b0;  
        DataMemRead   = 1'b0;  
        LEDWrite      = 1'b0;  
        SwitchReadEnable = 1'b0;  
  
        case (deviceSelect)  
            2'b00: begin // Data Memory (0-511)  
                DataMemWrite = writeEnable;  
                DataMemRead  = readEnable;  
            end  
            2'b01: begin // LEDs (512-767) - write only  
                LEDWrite = writeEnable;  
            end  
            2'b10: begin // Switches (768-1023) - read only  
                SwitchReadEnable = readEnable;  
            end  
            default: begin // 2'b11 - no device selected  
                DataMemWrite  = 1'b0;  
                DataMemRead   = 1'b0;  
                LEDWrite      = 1'b0;  
                SwitchReadEnable = 1'b0;  
            end  
        endcase  
    end  
endmodule
```



Testbench

module MemorySystem_th;

```
// -----
// DUT ports
// -----
reg    clk, rst;
reg [31:0] address;
reg    readEnable, writeEnable;
reg [31:0] writeData;
reg [15:0] switches;
wire [31:0] readData;
wire [15:0] leds;
// -----
// Hierarchical probes - visible in the waveform viewer
// -----
wire DataMemWrite  = DUT.decoder.DataMemWrite;
wire DataMemRead   = DUT.decoder.DataMemRead;
wire LEDWrite      = DUT.decoder.LEDWrite;
wire SwitchReadEnable =
DUT.decoder.SwitchReadEnable;

// -----
// DUT instantiation
// -----
addressDecoderTop DUT (
    .clk    (clk),
    .rst    (rst),
    .address (address),
    .readEnable (readEnable),
    .writeEnable (writeEnable),
    .writeData (writeData),
    .switches (switches),
    .readData (readData),
    .leds    (leds)
);

// -----
// Clock: 10 ns period (100 MHz)
// -----
initial clk = 0;
always #5 clk = ~clk;

// -----
// Pass / Fail counters
// -----
integer pass_cnt = 0;
integer fail_cnt = 0;

// -----
// TASK: check - compare expected vs actual and print result
// -----
task check;
    input [159:0] label;
    input [31:0] expected;
    input [31:0] actual;
    begin
        if (expected == actual) begin
            $display(" PASS %22s exp=%0h got=%0h",
                label, expected, actual);
            pass_cnt = pass_cnt + 1;
        end else begin
            $display(" FAIL %22s exp=%0h got=%0h <---",
                label, expected, actual);
            fail_cnt = fail_cnt + 1;
        end
    end
endtask

// -----
// TASK: do_write - synchronous write (one clock cycle)
// Drives on negedge so inputs are stable before posedge capture.
// -----
task do_write;
    input [31:0] addr;
    input [31:0] data;
    begin
        @(negedge clk);
        address = addr;
        writeData = data;
        writeEnable = 1'b1;
        readEnable = 1'b0;
        @(posedge clk); #1; // data captured on this rising
    end
endtask

// -----
// TASK: do_read - combinational read
// Asserts readEnable, waits 2 ns for combinational settle,
// caller samples signals, then deasserts after next posedge.
// -----
task do_read;
    input [31:0] addr;
    begin
        @(negedge clk);
        address = addr;
        readEnable = 1'b1;
        writeEnable = 1'b0;
        #2; // combinational paths settle (well within half-period)
        *** caller checks readData and decoder signals
    HERE ***
        @(posedge clk); #1;
        readEnable = 1'b0;
    end
endtask

// -----
// STIMULUS
// -----
initial begin
```

```
$display("-----");

// ----- Reset -----
rst    = 1'b1;
address = 32'd0;
readEnable = 1'b0;
writeEnable = 1'b0;
writeData = 32'd0;
switches = 16'd0;
repeat (4) @(posedge clk);
@(negedge clk);
rst = 1'b0;

// -----

// 1. DATA MEMORY - address[9:8] = 2'b00 (addresses 0-255)
// -----
$display("n[1] Data Memory - Writes");
do_write(32'd0, 32'hAAAA_BBBB);
do_write(32'd1, 32'h1234_5678);
do_write(32'd10, 32'hDEAD_BEEF);
do_write(32'd200, 32'hCAFE_F00D);
do_write(32'd255, 32'h0000_FFFF); // last address in DataMem range

$display("n[2] Data Memory - Read-Back");
do_read(32'd0);
check("DM[0]=AAAABBBB", 32'hAAAA_BBBB, readData);
check("DataMemRead hi", 32'd1, DataMemRead);
check("DataMemWrite lo", 32'd0, DataMemWrite);
check("LEDWrite lo", 32'd0, LEDWrite);
check("SwitchRdEn lo", 32'd0, SwitchReadEnable);

do_read(32'd1);
check("DM[1]=12345678", 32'h1234_5678, readData);

do_read(32'd10);
check("DM[10]=DEADBEEF", 32'hDEAD_BEEF, readData);

do_read(32'd200);
check("DM[200]=CAFEF00D", 32'hCAFE_F00D, readData);

do_read(32'd255);
check("DM[255]=0000FFFF", 32'h0000_FFFF, readData);

// -----

// 2. LED CONTROLLER - address[9:8] = 2'b01 (addresses 256-511)
// -----
$display("n[3] LED Controller - Writes");
do_write(32'd256, 32'h0000_A5A5); // first LED address
@(posedge clk); #1; // extra cycle: led_reg latches
check("LEDs=A5A5", 32'h0000_A5A5, {16'b0, leds});

do_write(32'd300, 32'h0000_5A5A);
@(posedge clk); #1;
check("LEDs=5A5A", 32'h0000_5A5A, {16'b0, leds});

do_write(32'd511, 32'h0000_1234); // last LED address
@(posedge clk); #1;
check("LEDs=1234", 32'h0000_1234, {16'b0, leds});

// Verify decoder exclusivity during an LED write
@(negedge clk);
address = 32'd400; // LED range
writeEnable = 1'b1;
readEnable = 1'b0;
#2;
check("LED: LW=1", 32'd1, LEDWrite);
check("LED: DmW=0", 32'd0, DataMemWrite);
check("LED: DmR=0", 32'd0, DataMemRead);
check("LED: SRE=0", 32'd0, SwitchReadEnable);
@(posedge clk); #1;
writeEnable = 1'b0;

// -----

// 3. SWITCH INTERFACE - address[9:8] = 2'b10 (addresses 512-767)
// -----
$display("n[4] Switch Interface - Reads");
switches = 16'hF0F0;
do_read(32'd512); // first Switch address -> [9:8]=10
check("SW[512]=F0F0", 32'h0000_F0F0, readData);
check("SwitchRdEn hi", 32'd1, SwitchReadEnable);
check("SW: DmR=0", 32'd0, DataMemRead);
check("SW: DmW=0", 32'd0, DataMemWrite);
check("SW: LW=0", 32'd0, LEDWrite);

switches = 16'h0F0F;
do_read(32'd600); // mid Switch range
check("SW[600]=0F0F", 32'h0000_0F0F, readData);
check("SRE hi (600)", 32'd1,
```

```
SwitchReadEnable);

switches = 16'hAAAA;
do_read(32'd767); // last Switch address
check("SW[767]=AAAA", 32'h0000_AAAA, readData);
check("SRE hi (767)", 32'd1, SwitchReadEnable);

switches = 16'h5555;
do_read(32'd512);
check("SW[512]=5555", 32'h0000_5555, readData);

switches = 16'hFFFF;
do_read(32'd700);
check("SW[700]=FFFF", 32'h0000_FFFF, readData);

// -----

// 4. UNUSED RANGE - address[9:8] = 2'b11 (addresses 768-1023)
// All decode outputs must remain low.
// -----
$display("n[5] Unused Address Range");

@(negedge clk);
address = 32'd768; // [9:8]=11 -> no device
readEnable = 1'b1;
writeEnable = 1'b0;
#2;
check("Unused: SRE=0", 32'd0, SwitchReadEnable);
check("Unused: DmR=0", 32'd0, DataMemRead);
check("Unused: LW=0", 32'd0, LEDWrite);
check("Unused: DmW=0", 32'd0, DataMemWrite);
check("Unused: rDat=0", 32'd0, readData);
@(posedge clk); #1;
readEnable = 1'b0;

@(negedge clk);
address = 32'd1023; // [9:8]=11 -> no device
writeEnable = 1'b1;
readEnable = 1'b0;
#2;
check("Unused1023:DmW=0", 32'd0, DataMemWrite);
check("Unused1023:LW=0", 32'd0, LEDWrite);
@(posedge clk); #1;
writeEnable = 1'b0;

// -----

// 5. DECODER EXCLUSIVITY - no enables overlap for any device
// -----
$display("n[6] Decoder Exclusivity Sweep");

// DataMem write
@(negedge clk); address=32'd128; writeEnable=1; readEnable=0; #2;
check("Excl DmW=1", 32'd1, DataMemWrite);
check("Excl DmR=0", 32'd0, DataMemRead);
check("Excl LW=0", 32'd0, LEDWrite);
check("Excl SRE=0", 32'd0, SwitchReadEnable);
@(posedge clk); #1; writeEnable=0;

// DataMem read
@(negedge clk); address=32'd128; readEnable=1; writeEnable=0; #2;
check("Excl DmR=1", 32'd1, DataMemRead);
check("Excl DmW=0", 32'd0, DataMemWrite);
check("Excl LW=0", 32'd0, LEDWrite);
check("Excl SRE=0", 32'd0, SwitchReadEnable);
@(posedge clk); #1; readEnable=0;

// LED write
@(negedge clk); address=32'd350; writeEnable=1; readEnable=0; #2;
check("Excl LW=1", 32'd1, LEDWrite);
check("Excl DmW=0", 32'd0, DataMemWrite);
check("Excl DmR=0", 32'd0, DataMemRead);
check("Excl SRE=0", 32'd0, SwitchReadEnable);
@(posedge clk); #1; writeEnable=0;

// Switch read
@(negedge clk); address=32'd650; readEnable=1; writeEnable=0; #2;
check("Excl SRE=1", 32'd1, SwitchReadEnable);
check("Excl DmW=0", 32'd0, DataMemWrite);
check("Excl DmR=0", 32'd0, DataMemRead);
check("Excl LW=0", 32'd0, LEDWrite);
@(posedge clk); #1; readEnable=0;

// -----

// Summary
// -----
repeat(4) @(posedge clk);

$display("n-----");
$display(" TOTAL: PASS = %0d FAIL = %0d", pass_cnt, fail_cnt);
if (fail_cnt == 0)
    $display(" *** All tests passed! ***");
else
    $display(" *** %0d test(s) FAILED - review above ****", fail_cnt);

$display("n-----");
$stop;
end
```

endmodule

Simulation waveforms showing correct operation\

DATA MEMORY SECTION:

DATA MEMORY WRITE (40ns-100ns):

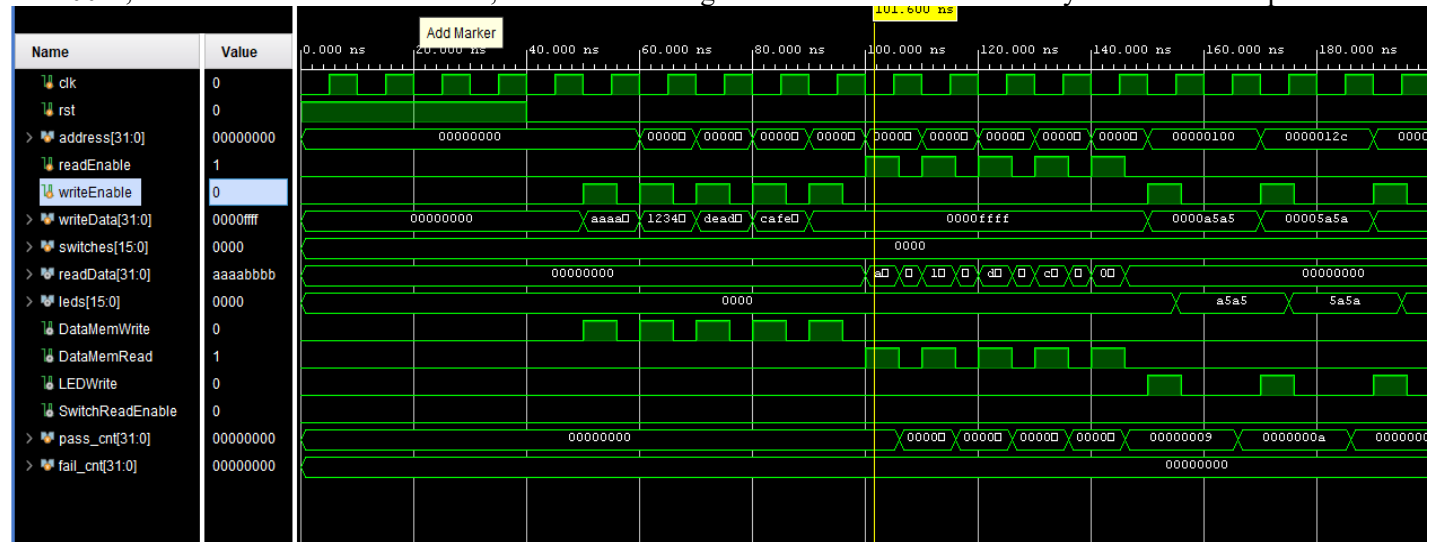
DATA MEMORY READ (100ns- 140ns)

Waveform explanations:

Data Memory section — around t=40-140 ns:

- `writeEnable` goes high → `DataMemWrite` should pulse high
- `readEnable` goes high → `DataMemRead` high, `readData` shows `DEADBEEF`, `12345678`, etc.

AT 100ns, all the values that were stored; will start showing at READ DATA. Where the yellow marker is places

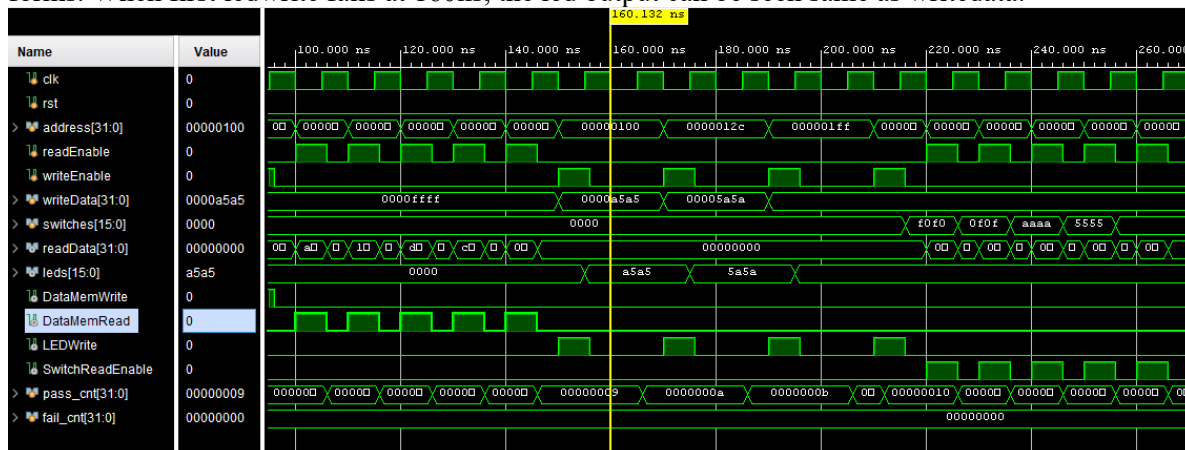


LED SECTION:

LED section — around t=150-220 ns:

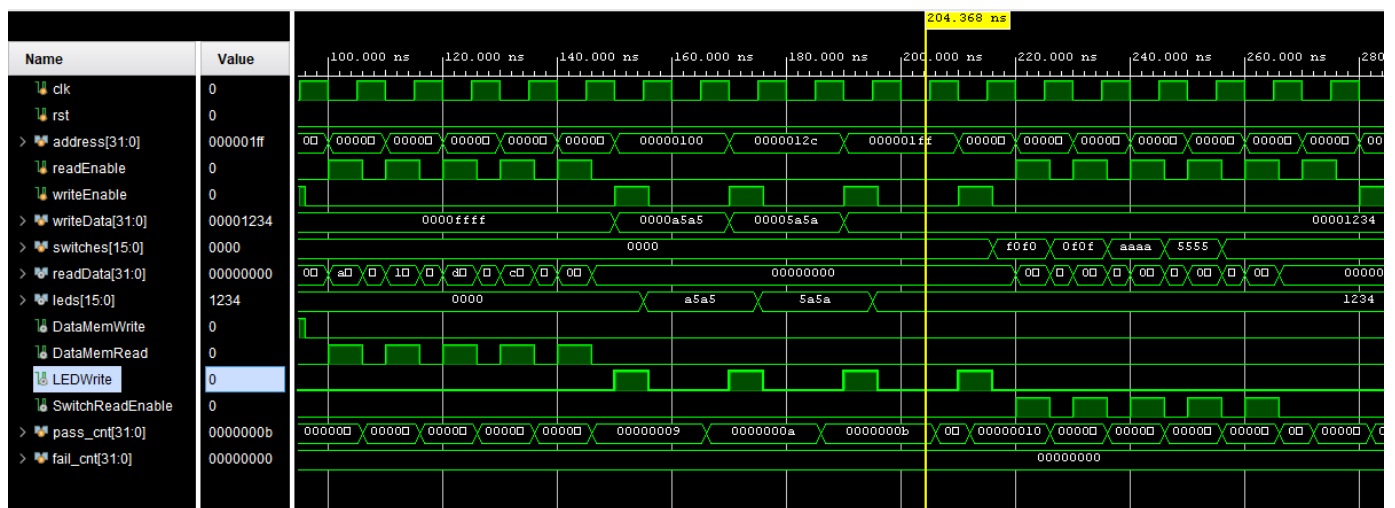
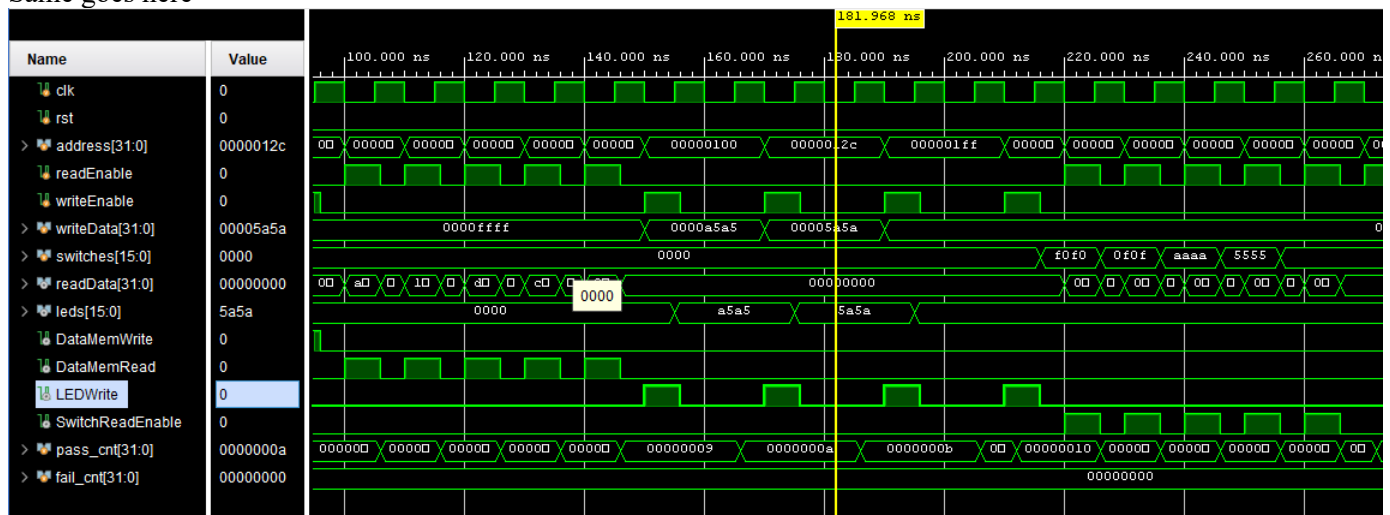
- `LEDWrite` pulses high on each write
- `leds` output changes one clock later (registered)

As WRITE DATA has some value, so when led write rises, the value is shown on leds. After it falls back. As seen in the waveforms. When first ledwrite falls at 160ns, the led output can be seen same as writedata.





Same goes here



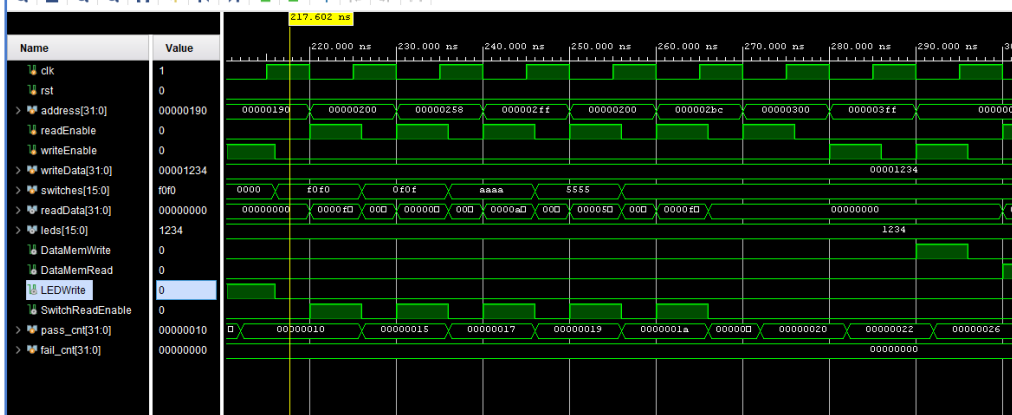
SWITCH SECTION:

Switch section — around $t=230-320$ ns:

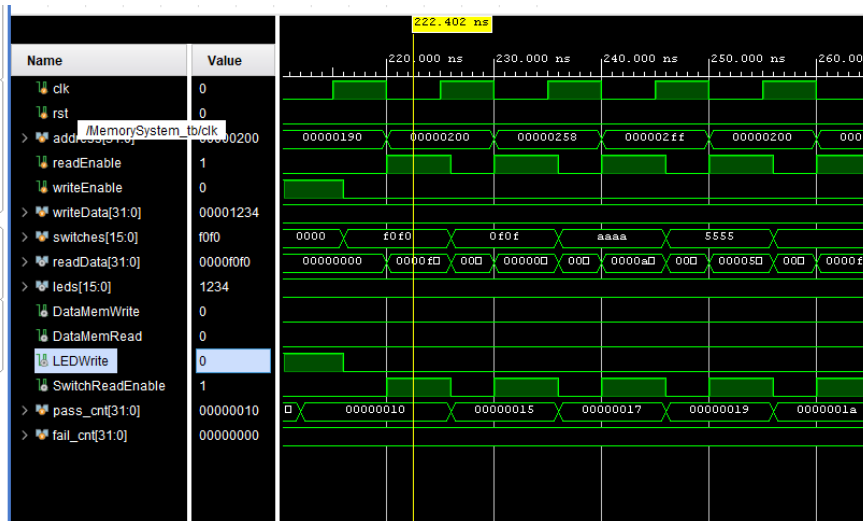
- **SwitchReadEnable** goes HIGH ← this is what was broken before
- **readData** shows **0000F0F0**, **0000F0F**, **0000AAAA** matching switch inputs

As we put input in switches, then when the READDATA turns 1, it basically replicates the value of switches, as can be seen in wave forms.

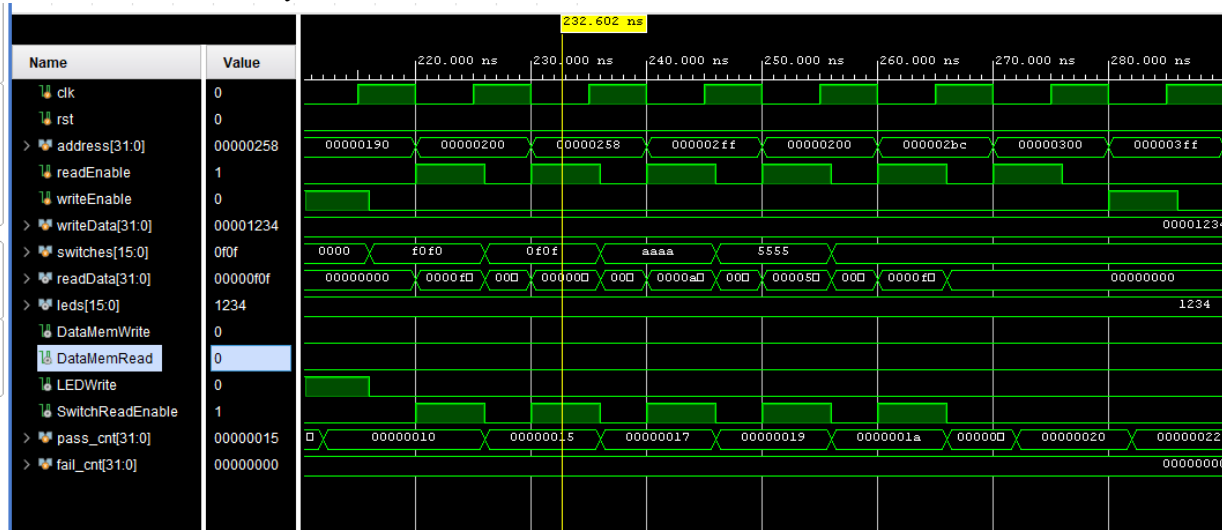
As here in this wave form we can see that switch read enable was zero. Readdata, did not get the value which was in switches.

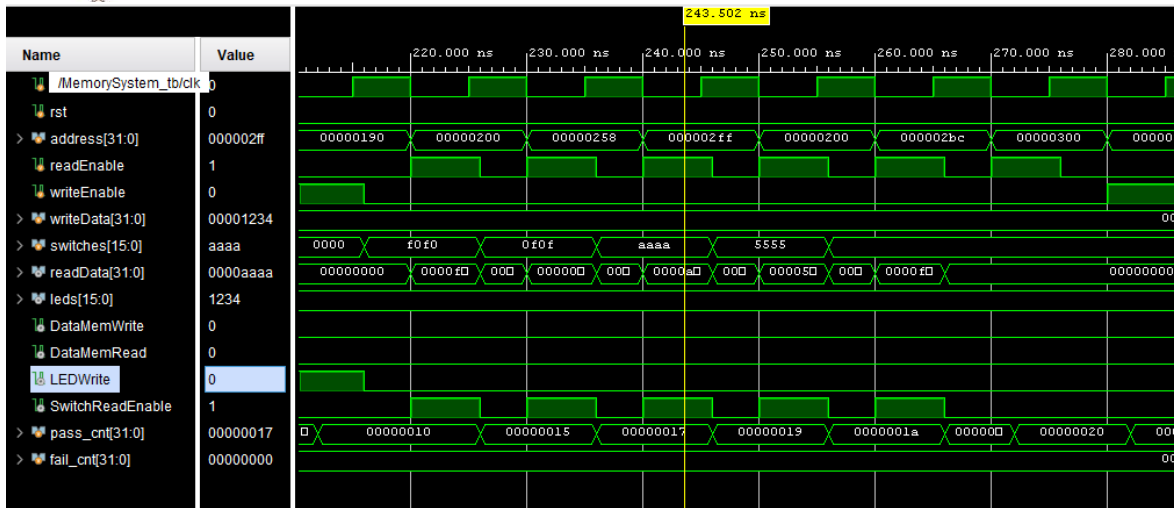


Now when the switchreadenable becomes 1. It reads the value of switches as can be seen in the following waveforms.

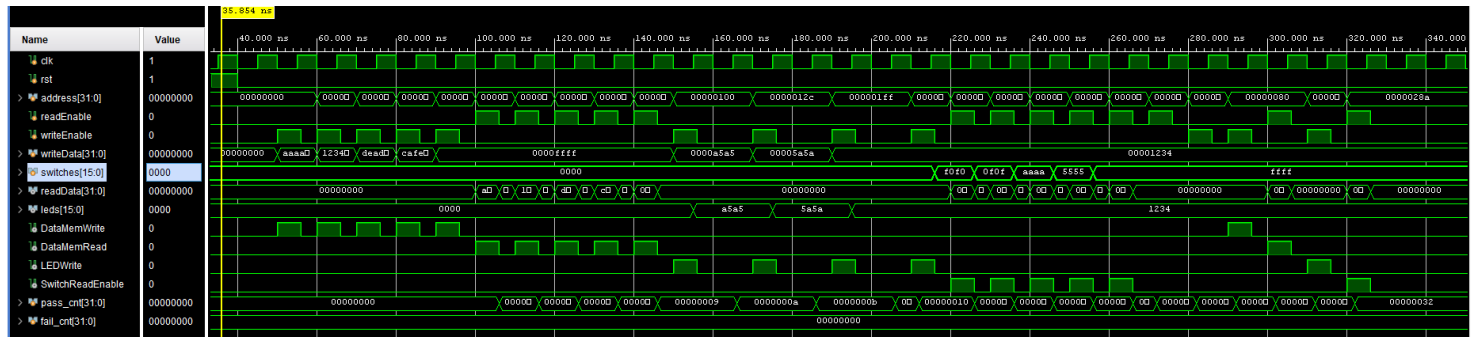


Rest is also the same way



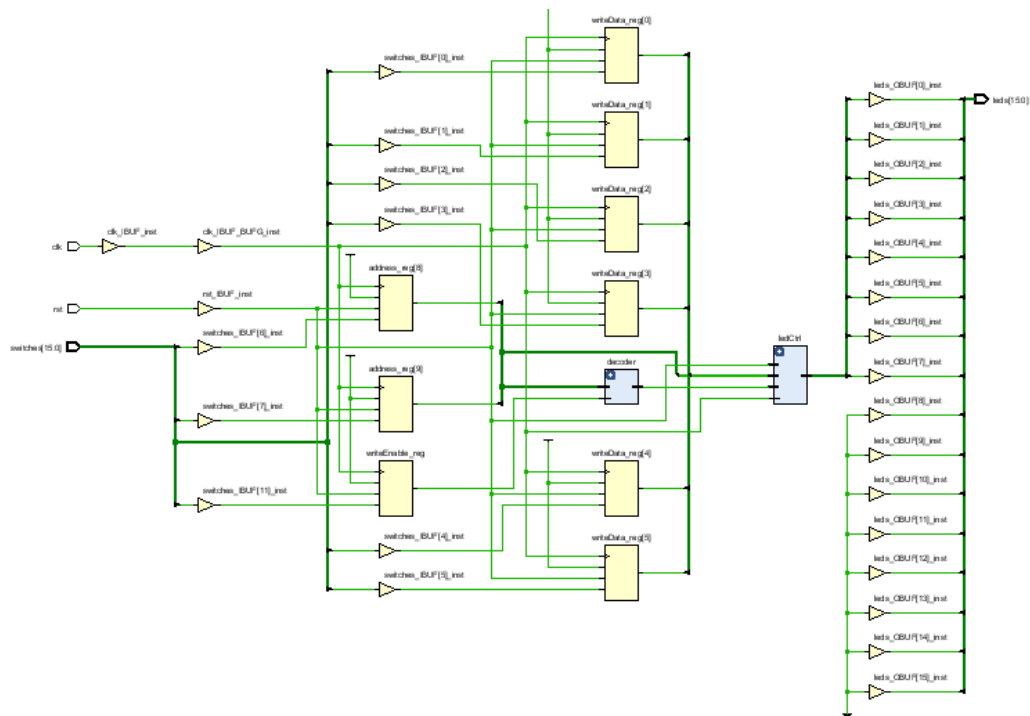


Picture of full waveform of test bench



Task 3: FPGA Synthesis and RTL Analysis

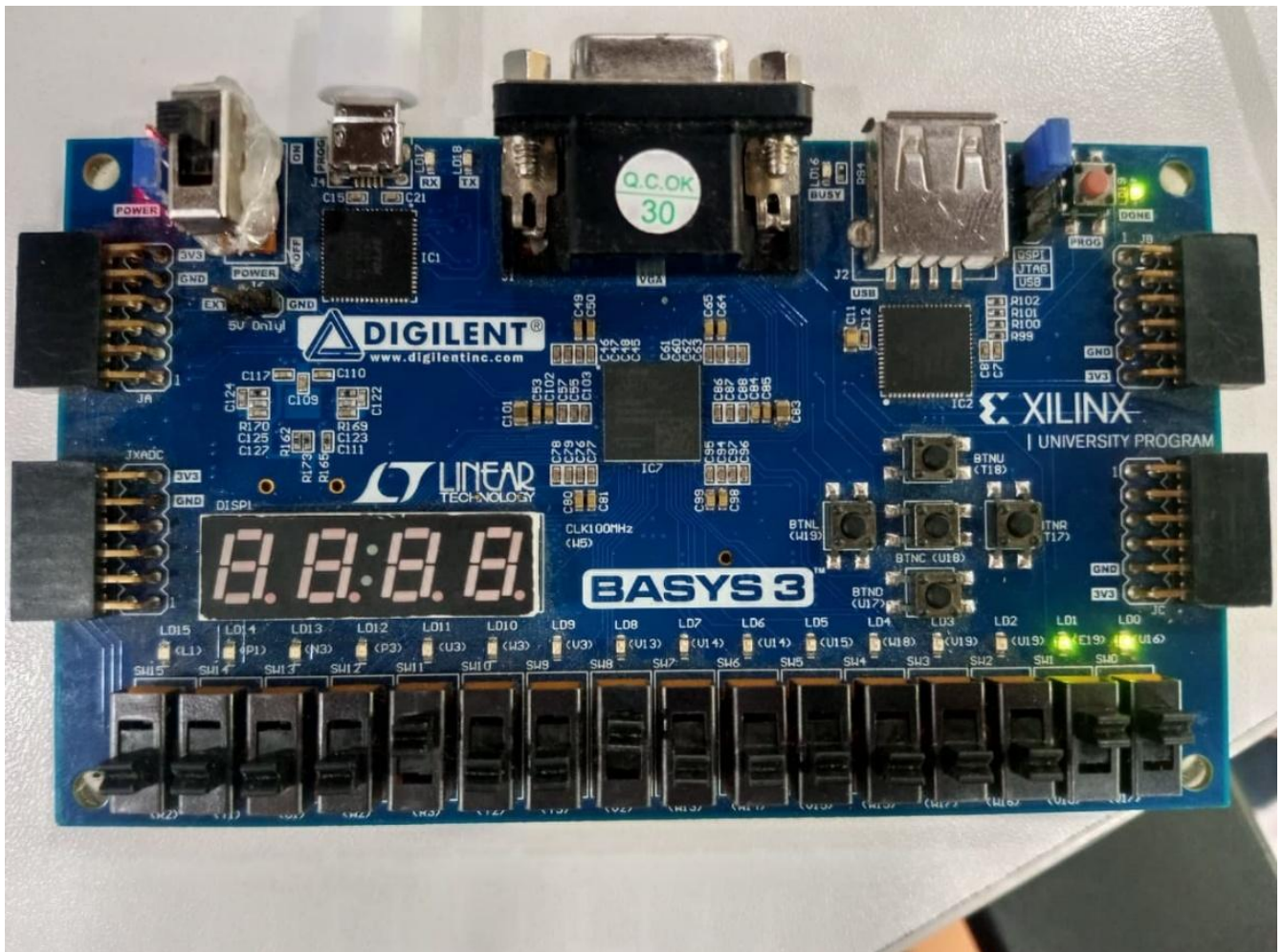
RTL schematic screenshots (register file array, decoder, FSM, ALU connections)



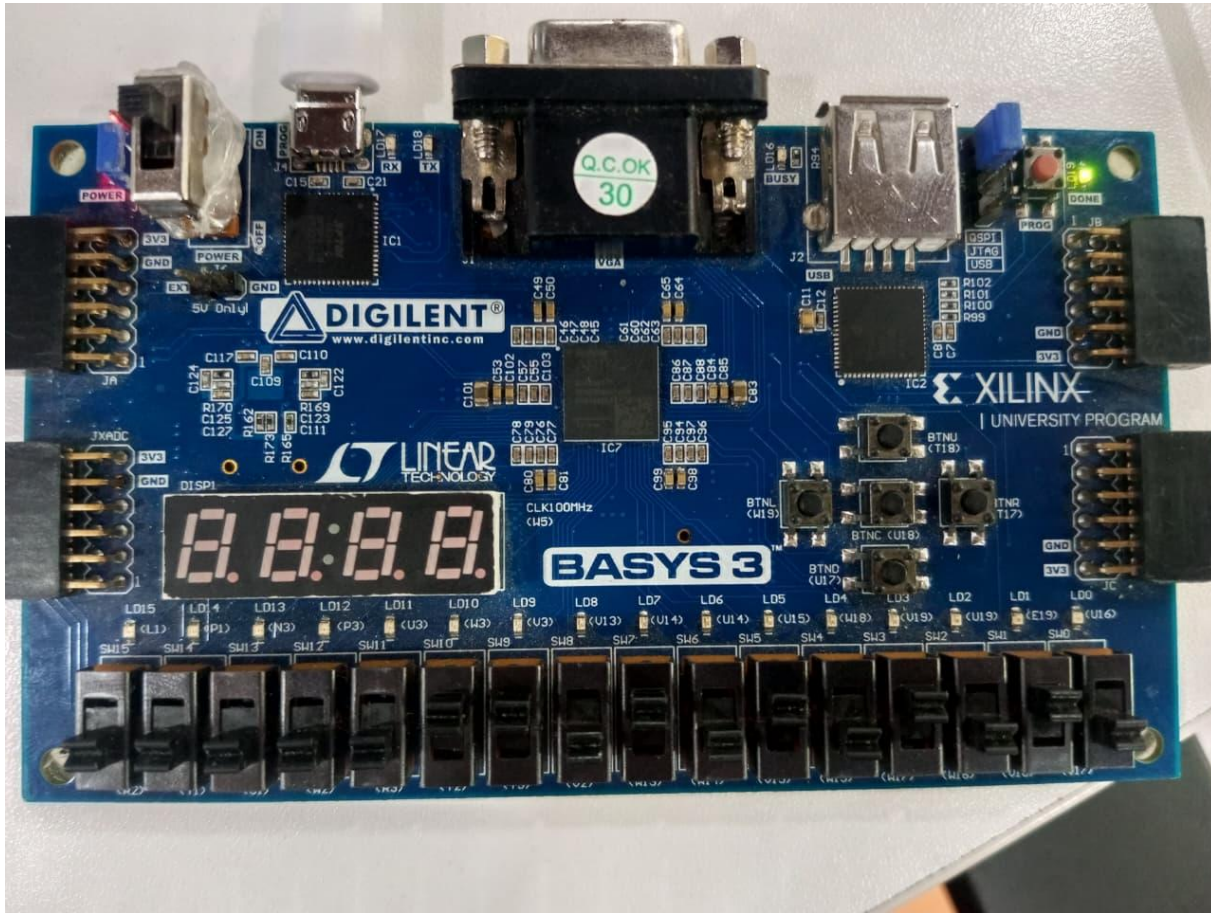
Demonstration of FPGA operation

SW0 → writeData bit0 → LD0
 SW1 → writeData bit1 → LD1
 SW2 → writeData bit2 → LD2
 SW3 → writeData bit3 → LD3
 SW4 → writeData bit4 → LD4
 SW5 → writeData bit5 → LD5
 SW6 → writeData bit6 → LD6
 SW7 → writeData bit7 → LD7
 SW8 → address[8] (device select LSB)
 SW9 → address[9] (device select MSB)
 SW10 → readEnable
 SW11 → writeEnable
 SW12 → unused
 SW13 → unused
 SW14 → unused
 SW15 → unused

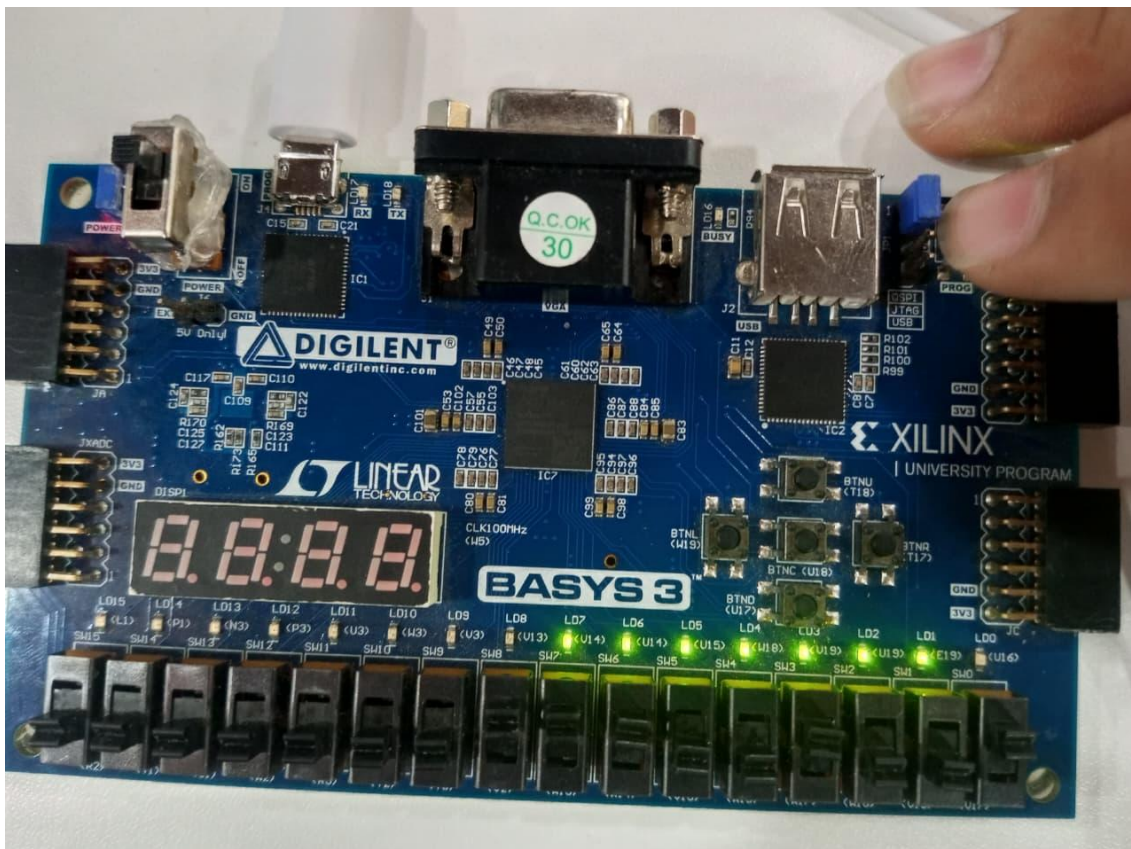
Write data



Read data from switches. No led will turn on



Write data holds value when write enable is off. And switches are even changed





All LEDs will be OFF when:

Reset; press btnC anytime

SW11=DOWN: writeEnable off, nothing gets written

SW8=DOWN, SW9=DOWN, SW11=UP: memory region, LEDWrite never asserts

SW8=DOWN, SW9=UP, SW11=UP: switch region, LEDWrite never asserts

SW8=UP, SW9=UP, SW11=UP: invalid region 11, no device selected

SW8=UP, SW9=DOWN, SW11=UP, SW0-SW7=all DOWN: correct region but writing 0x00 to LEDs

Power on before any switch touched: register initializes to 0



Assessment Rubric

Lab 8: Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/15	-
	Task 3	/10	/15	-
Total Points		/30	/30	/20
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.